Análisis comparativo del desempeño empírico entre Insertion Sort y Merge Sort en C++ según patrones de datos

José Alonso Nina Arocutipa
jninaa@unjbg.edu.pe
Escuela Profesional de Ingeniería en Informática y Sistemas, Universidad Nacional Jorge Basadre Grohmann
Tacna, Perú

Resumen

En el presente estudio se evalúa y compara el desempeño empírico de dos algoritmos de ordenamiento interno representativos: Insertion Sort (método directo cuadrático) y Merge Sort (método logarítmico basado en divide y vencerás). Se diseñó e implementó un protocolo experimental riguroso en C++ con instrumentación para medir tiempo de ejecución en milisegundos (steady_clock), número de comparaciones, intercambios/movimientos y profundidad máxima de pila. Las pruebas abarcaron tres tamaños de entrada (n en 1,000, 10,000, 100,000) y cinco patrones de distribución inicial (aleatorio, ascendente, descendente, casi ordenado con 5% de permutaciones y con valores duplicados), ejecutando 50 repeticiones por condición. Los resultados confirman la superioridad asintótica de Merge Sort (O(n \log n)) en arreglos de gran escala, mientras que Insertion Sort exhibe una eficiencia notable (O(n)) en conjuntos casi ordenados de menor escala.

Palabras clave: Algoritmos de ordenamiento, Insertion Sort, Merge Sort, Complejidad computacional, C++, Rendimiento empírico.

Abstract

In this study, the empirical performance of two representative internal sorting algorithms is evaluated and compared: Insertion Sort (a quadratic direct method) and Merge Sort (a logarithmic divide-and-conquer method). A rigorous experimental protocol was designed and implemented in C++ with instrumentation to measure execution time in milliseconds (steady_clock), number of comparisons, swaps/movements, and maximum stack depth. Experiments covered three input sizes (n in 1,000, 10,000, 100,000) and five initial distribution patterns (random, ascending, descending, nearly sorted with 5% permutations, and duplicated values), running 50 repetitions per condition. Results confirm the asymptotic superiority of Merge Sort (O(n \log n)) in large-scale arrays, whereas Insertion Sort exhibits remarkable efficiency (O(n)) in nearly sorted, smaller-scale sets.

Keywords: Sorting algorithms, Insertion Sort, Merge Sort, Computational complexity, C++, Empirical performance.

1. Introducción

El ordenamiento de datos en memoria principal es una operación fundamental en la ciencia de la computación. Los algoritmos de ordenamiento se clasifican comúnmente según su complejidad temporal teórica en algoritmos cuadráticos (O(n^2)) y logarítmicos (O(n \log n)). Mientras que el análisis asintótico tradicional provee cotas superiores e inferiores teóricas, el rendimiento real en entornos de ejecución depende de múltiples factores prácticos como la arquitectura del procesador, el comportamiento del caché de memoria y el patrón inicial de los datos de entrada.

El objetivo de este trabajo es realizar un estudio comparativo y empírico entre dos algoritmos estables de ordenamiento interno: Insertion Sort (Inserción directa) y Merge Sort (Mezcla). Se busca identificar los puntos de inflexión (*cross-over points*) donde un método supera al otro en función del tamaño de los datos y su disposición inicial. La hipótesis descriptiva plantea que, si bien Merge Sort dominará en arreglos aleatorios e invertidos de tamaño mediano y grande debido a su complejidad O(n \log n), Insertion Sort mantendrá un desempeño superior o competitivo en arreglos de escala reducida y en datos casi ordenados debido a su baja sobrecarga en memoria auxiliar y reducido número de operaciones en su mejor caso (O(n)).

2. Metodología Experimental

Para garantizar la reproducibilidad del experimento, se implementó una suite de benchmarking instrumentada en C++17.

2.1 Selección de Algoritmos y Variantes

- **Insertion Sort (Canónico):** Algoritmo iterativo in-situ con complejidad espacial O(1). Presenta un comportamiento de O(n) en el mejor caso (arreglo ordenado) y O(n^2) en el peor y promedio caso.
- **Merge Sort (Canónico Recursivo):** Algoritmo recursivo basado en divide y vencerás con complejidad teórica O(n \log n) en todos los casos. Requiere una memoria auxiliar de O(n) para la fase de mezcla y genera una pila de llamadas con profundidad máxima O(\log n).

Ambos algoritmos conservan el orden relativo de elementos con claves iguales, garantizando la propiedad de estabilidad.

2.2 Escenarios de Entrada y Tamaños

Se evaluaron tres tamaños de entrada: n en 1,000, 10,000, 100,000. Para cada tamaño n, se generaron instancias con 5 patrones de datos únicos usando el generador pseudoaleatorio mt19937 con semilla fija (2026):

- **Aleatorio uniforme:** Enteros distribuidos en rango amplio.
- **Ordenado ascendente:** Arreglo estricta y previamente ordenado.
- **Ordenado descendente (Peor caso para Inserción):** Arreglo totalmente invertido.

- **Casi ordenado:** Arreglo previamente ascendente con un 5% de sus elementos permutados aleatoriamente.
- **Con duplicados:** Valores restringidos al rango [1, 15] para forzar múltiples colisiones de elementos idénticos.

2.3 Métricas Registradas

1. **Tiempo de ejecución (ms):** Capturado con std::chrono::steady_clock, excluyendo tiempos de E/S y generación de instancias.
2. **Comparaciones:** Contador acumulativo de evaluaciones entre pares de elementos.
3. **Intercambios / Movimientos:** Conteo de asignaciones y desplazamientos de datos.
4. **Profundidad de recursión:** Registro de la máxima profundidad alcanzada en la pila para Merge Sort.

Se realizaron 50 ejecuciones independientes por cada combinación de algoritmo, patrón y tamaño.

3. Resultados

A continuación se presentan los promedios y desviaciones estándar obtenidos en las mediciones empíricas tras 50 repeticiones.

Tabla 1
Comparativa de Tiempo de Ejecución (ms) y Operaciones Algorítmicas (Promedio)

Algoritmo	Tamaño (n)	Patrón	Tiempo Medido (ms)	Comparaciones Promedio	Intercambios / Movimientos	Profundidad Recursión
Insertion Sort	1,000	Aleatorio	1.12	248,512	247,512	0
Merge Sort	1,000	Aleatorio	0.18	8,712	9,968	10
Insertion Sort	1,000	Casi Ordenado	0.04	3,210	2,210	0
Merge Sort	1,000	Casi Ordenado	0.17	8,650	9,968	10
Insertion Sort	10,000	Aleatorio	115.40	24,981,200	24,971,200	0
Merge Sort	10,000	Aleatorio	2.10	120,450	133,616	14
Insertion Sort	10,000	Ascendente	0.02	9,999	0	0
Merge Sort	10,000	Ascendente	1.85	69,000	133,616	14

Algoritmo	Tamaño (n)	Patrón	Tiempo Medido (ms)	Comparaciones Promedio	Intercambios / Movimientos	Profundidad Recursión
Insertion Sort	100,000	Aleatorio	11,850.20	2,499,120,000	2,499,020,000	0
Merge Sort	100,000	Aleatorio	24.50	1,536,400	1,668,928	17

4. Discusión

Los datos empíricos ratifican la teoría de complejidad asintótica. Para tamaños de arreglo grandes (n = 100,000), Merge Sort demuestra ser drásticamente más eficiente que Insertion Sort en arreglos aleatorios (24.50 ms frente a 11,850.20 ms). Esta diferencia abrumadora radica en el número total de comparaciones e intercambios, los cuales crecen de forma cuadrática O(n^2) en Inserción y de forma logarítmica O(n \log n) en Merge Sort.

Sin embargo, en el escenario casi ordenado o ascendente con n \le 10,000, Insertion Sort supera significativamente a Merge Sort. Para n = 10,000 en patrón ascendente, Insertion Sort requirió únicamente 0.02 ms frente a los 1.85 ms de Merge Sort. Esto se explica porque Insertion Sort no realiza ningún intercambio y ejecuta exactamente n-1 comparaciones, mientras que Merge Sort continúa dividiendo recursivamente el arreglo y reservando memoria auxiliar sin aprovechar el orden previo de los datos.

Respecto a la estabilidad y el uso de recursos, Merge Sort incurre en una sobrecarga constante debido a la asignación dinámica de arreglos auxiliares L y R y el mantenimiento de la pila recursiva (alcanzando una profundidad máxima de 17 llamadas para n = 100,000, equivalente a \lceil \log_2(100,000) \rceil).

5. Conclusiones

1. Merge Sort es el algoritmo superior para conjuntos de datos grandes o desordenados, demostrando un comportamiento estable y predecible O(n \log n) a costa de un consumo de memoria auxiliar de O(n).
2. Insertion Sort es sumamente eficaz e imbatible en conjuntos de datos pequeños o datos casi totalmente ordenados, operando in-situ con memoria O(1).
3. La evaluación del patrón de datos inicial es crucial al elegir un algoritmo de ordenamiento en la práctica, respaldando el uso de enfoques híbridos como Timsort (que combina Inserción y Mezcla en la práctica profesional).

Referencias

- Murillo, J., & Caamaño, S. (2013). Comparación entre algoritmos recursivos e iterativos y su medición en términos de eficiencia. *Uniciencia*, 27(1), 341-350.

- Salgado, M., Pérez, J., & Albarrán, S. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia E Ingeniería*, 43(03), 273-280.
- Terán Pozo, E. J. (2021). Influencia de los Algoritmos Genéticos en la generación de horarios en la Unidad Educativa Ana Luisa Leoro. *Revista COMTEL*, 1-6.